# Effective C++: Comprehensive Guide

Ajay Kumar

August 2026

## Contents

# 1 Chapter 1: Accustoming Yourself to C++

## 1.1 Item 1: View C++ as a Federation of languages

**Concept**: C++ is best understood as a federation of four sub-languages: - **C**: Blocks, statements, preprocessor, built-in data types, arrays, pointers - **Object-Oriented C++**: Classes, encapsulation, inheritance, polymorphism, virtual functions - **Template C++**: Generic programming, template metaprogramming - **The STL**: Containers, iterators, algorithms, function objects

**Key Insight**: Effective programming requires understanding which sub-language you're working in, as rules and best practices can vary.

**Example**:

```cpp
// C sub-language: Pass by value is efficient for built-in types
void processValue(int x) {
    // Working with primitive type
}

// Object-Oriented C++: Pass by reference-to-const is preferred
void processWidget(const Widget& w) {
    // Working with user-defined type
}

// Template C++: Generic programming with type parameters
template<typename T>
void process(const T& param) {
    // Behavior depends on T
}

// STL: Iterator-based algorithms
std::vector<int> vec = {1, 2, 3, 4, 5};
std::for_each(vec.begin(), vec.end(), [](int x) {
    std::cout << x << " ";
});
```

## 1.2 Item 2: Prefer consts, enums, and inlines to #defines

**Concept**: Use the compiler instead of the preprocessor whenever possible.

**Rationale**: - `#define` doesn't respect scope and provides no type safety - Constants and inline functions provide better error messages and debugging - Compiler can optimize better with actual language constructs

**Example**:

```cpp
// [BAD] BAD: Using #define
#define ASPECT_RATIO 1.653
#define CALL_WITH_MAX(a, b) ((a) > (b) ? (a) : (b))

// [GOOD] GOOD: Using const
const double AspectRatio = 1.653;

// [GOOD] GOOD: Class-specific constant
class GamePlayer {
private:
    static const int NumTurns = 5;  // Declaration
    int scores[NumTurns];           // Use of constant
};

// [GOOD] GOOD: When you need the address or if compiler insists
const int GamePlayer::NumTurns;     // Definition

// [GOOD] GOOD: Enum hack (compile-time constant)
class GamePlayer2 {
private:
    enum { NumTurns = 5 };
    int scores[NumTurns];
};

// [GOOD] GOOD: Inline function instead of macro
template<typename T>
inline const T& callWithMax(const T& a, const T& b) {
    return a > b ? a : b;
}
```

```cpp
// Why inline is better than macro:
int a = 5, b = 0;
// CALL_WITH_MAX(++a, b);     // a incremented twice!
callWithMax(++a, b);          // a incremented once (predictable)
```

### 1.3 Item 3: Use const whenever possible

**Concept**: The `const` keyword is powerful for expressing design intent and catching errors at compile time.

**Applications**: - Variables - Pointers and references - Function parameters - Function return types - Member functions

**Example**:

```cpp
// Const with pointers
const char* p1 = "Hello";           // Pointer to const char
char* const p2 = "World";           // Const pointer to char
const char* const p3 = "!";         // Const pointer to const char

// STL iterators
std::vector<int> vec;
const std::vector<int>::iterator iter = vec.begin();  // Like T* const
*iter = 10;     // OK
// ++iter;      // Error

std::vector<int>::const_iterator cIter = vec.begin(); // Like const T*
// *cIter = 10; // Error
++cIter;        // OK

// Const return values prevent errors
class Rational {
public:
    // ...
};

const Rational operator*(const Rational& lhs, const Rational& rhs);

// Prevents this error:
Rational a, b, c;
// (a * b) = c;  // Error! Can't assign to const
// if (a * b = c) // Error! Typo caught by compiler

// Const member functions
class TextBlock {
public:
    const char& operator[](std::size_t position) const {
        // const version for const objects
        return text[position];
    }

    char& operator[](std::size_t position) {
        // non-const version for non-const objects
        return text[position];
    }

private:
    std::string text;
};

// Usage:
void print(const TextBlock& ctb) {
```

```
    std::cout << ctb[0];  // Calls const operator[]
    // ctb[0] = 'x';       // Error! Can't modify
}

TextBlock tb("Hello");
tb[0] = 'x';              // OK: calls non-const operator[]
std::cout << tb[0];       // OK: calls non-const operator[]

const TextBlock ctb("World");
std::cout << ctb[0];      // OK: calls const operator[]
// ctb[0] = 'x';          // Error!
```

**Bitwise const vs. Logical const**:

```
class CTextBlock {
public:
    // Bitwise const: doesn't modify any member bits
    char& operator[](std::size_t position) const {
        return pText[position];  // Legal but questionable
    }

private:
    char* pText;
};

// Problem with bitwise const:
const CTextBlock cctb("Hello");
char* pc = &cctb[0];
*pc = 'J';  // Modifies "const" object!

// Solution: Logical const with mutable
class CTextBlock2 {
public:
    std::size_t length() const {
        if (!lengthIsValid) {
            textLength = std::strlen(pText);  // OK: mutable member
            lengthIsValid = true;             // OK: mutable member
        }
        return textLength;
    }

private:
    char* pText;
    mutable std::size_t textLength;
    mutable bool lengthIsValid;
};
```

**Avoiding Duplication in const and non-const Member Functions**:

```cpp
class TextBlock {
public:
    const char& operator[](std::size_t position) const {
        // Bounds checking
        // Log access
        // Verify data integrity
        return text[position];
    }

    char& operator[](std::size_t position) {
        // Implement in terms of const version to avoid duplication
        return const_cast<char&>(
            static_cast<const TextBlock&>(*this)[position]
        );
    }

private:
    std::string text;
};
```

### 1.4 Item 4: Make sure objects are initialized before they're used

**Concept**: Reading uninitialized values yields undefined behavior. Always initialize your objects.

**Rules**: 1. Manually initialize objects of built-in types 2. Use member initialization lists in constructors 3. Understand the order of initialization across translation units

**Example**:

```cpp
// [BAD] BAD: Reading uninitialized values
int x;
std::cout << x;  // Undefined behavior!

// [GOOD] GOOD: Initialize built-in types
int x = 0;
const char* text = "A C-style string";
double d;
std::cin >> d;  // Assignment before use

// Constructor initialization
class PhoneNumber { /* ... */ };
class ABEntry {
public:
    // [BAD] BAD: Using assignment in constructor body
    ABEntry(const std::string& name, const std::string& address,
            const std::list<PhoneNumber>& phones) {
        theName = name;        // Assignment, not initialization
        theAddress = address;
        thePhones = phones;
        numTimesConsulted = 0;
    }

private:
    std::string theName;
    std::string theAddress;
    std::list<PhoneNumber> thePhones;
    int numTimesConsulted;
};

// [GOOD] GOOD: Using member initialization list
class ABEntry2 {
```

```
public:
    ABEntry2(const std::string& name, const std::string& address,
             const std::list<PhoneNumber>& phones)
        : theName(name),                    // Initialization
          theAddress(address),
          thePhones(phones),
          numTimesConsulted(0)
    {}

    // Even better for default constructor
    ABEntry2()
        : theName(),           // Default-construct
          theAddress(),
          thePhones(),
          numTimesConsulted(0) // Explicit initialization
    {}

private:
    std::string theName;
    std::string theAddress;
    std::list<PhoneNumber> thePhones;
    int numTimesConsulted;
};
```

**Why Member Initialization List is Better**:

```
class Complex {
public:
    // Assignment version: Default constructor + assignment
    Complex(double r, double i) {
        real = r;       // 1. Default-construct real
        imag = i;       // 2. Assign to real
    }                   // Less efficient

    // Initialization list: Direct construction
    Complex(double r, double i)
        : real(r),      // Directly construct with value
          imag(i)
    {}                  // More efficient

private:
    double real;
    double imag;
};
```

**Initialization Order**:

```cpp
class FileSystem {
public:
    std::size_t numDisks() const;
};

extern FileSystem tfs;  // Object for clients to use

// In another file:
class Directory {
public:
    Directory() {
        std::size_t disks = tfs.numDisks();  // Use tfs
    }
};

Directory tempDir;  // Problem: tfs might not be initialized yet!
```

**Solution: Local Static Objects (Singleton Pattern)**:

```cpp
class FileSystem { /* ... */ };

FileSystem& tfs() {  // Replace non-local static object with function
    static FileSystem fs;  // Local static object
    return fs;
}

class Directory {
public:
    Directory() {
        std::size_t disks = tfs().numDisks();  // Now safe!
    }
};

Directory& tempDir() {  // Also use function
    static Directory td;
    return td;
}
```

---

# 2 Chapter 2: Constructors, Destructors, and Assignment Operators

## 2.1 Item 5: Know what functions C++ silently writes and calls

**Concept**: The compiler will automatically generate these functions if you don't declare them: - Default constructor (if no constructors are declared) - Copy constructor - Copy assignment operator - Destructor

**Example**:

```cpp
// Empty class
class Empty {};

// Equivalent to:
class Empty {
public:
    Empty() { }                                 // Default constructor
    Empty(const Empty& rhs) { }                 // Copy constructor
    ~Empty() { }                                // Destructor
    Empty& operator=(const Empty& rhs) { }      // Copy assignment operator
};

// Usage:
Empty e1;        // Default constructor
Empty e2(e1);    // Copy constructor
e2 = e1;         // Copy assignment operator
// e1 destroyed   // Destructor

// Generated copy constructor example:
class NamedObject {
public:
    NamedObject(const std::string& name, int value)
        : nameValue(name), objectValue(value) {}

private:
    std::string nameValue;
    int objectValue;
};

// Compiler-generated copy constructor does this:
NamedObject no1("Smallest Prime Number", 2);
NamedObject no2(no1);  // Copy constructor
// nameValue is copy-constructed from no1.nameValue
// objectValue is copied from no1.objectValue
```

**When Compiler Won't Generate Copy Assignment**:

```cpp
class NamedObject {
public:
    NamedObject(std::string& name, int value)
        : nameValue(name), objectValue(value) {}

private:
    std::string& nameValue;     // Reference member
    const int objectValue;      // Const member
};

std::string newDog("Persephone");
std::string oldDog("Satch");

NamedObject p(newDog, 2);
NamedObject s(oldDog, 36);

p = s;  // Error! Compiler won't generate copy assignment
        // Can't rebind reference or modify const member
```

## 2.2 Item 6: Explicitly disallow functions you do not want

**Concept**: Prevent copying when it doesn't make sense for your class.

**C++03 Approach**:

```
class HomeForSale {
public:
    // ...

private:
    HomeForSale(const HomeForSale&);              // Declared but not defined
    HomeForSale& operator=(const HomeForSale&);   // Declared but not defined
};

// Attempting to copy:
HomeForSale h1;
HomeForSale h2;
// HomeForSale h3(h1);  // Error: copy constructor is private
// h1 = h2;             // Error: copy assignment is private

// Base class approach for reusability:
class Uncopyable {
protected:
    Uncopyable() {}
    ~Uncopyable() {}

private:
    Uncopyable(const Uncopyable&);
    Uncopyable& operator=(const Uncopyable&);
};

class HomeForSale2 : private Uncopyable {
    // No need to declare copy operations
};
```

**C++11 and Later Approach**:

```
class HomeForSale {
public:
    // ...
    HomeForSale(const HomeForSale&) = delete;
    HomeForSale& operator=(const HomeForSale&) = delete;
};

// Even better: delete move operations too
class NoCopy {
public:
    NoCopy() = default;
    NoCopy(const NoCopy&) = delete;
    NoCopy& operator=(const NoCopy&) = delete;
    NoCopy(NoCopy&&) = delete;
    NoCopy& operator=(NoCopy&&) = delete;
};
```

## 2.3 Item 7: Declare destructors virtual in polymorphic base classes

**Concept**: When deleting a derived class object through a base class pointer, the base class must have a virtual destructor to avoid undefined behavior.

**Example**:

```cpp
// [BAD] BAD: Non-virtual destructor
class TimeKeeper {
public:
    TimeKeeper();
    ~TimeKeeper();  // Non-virtual!
    // ...
};

class AtomicClock : public TimeKeeper { /* ... */ };
class WaterClock : public TimeKeeper { /* ... */ };

TimeKeeper* ptk = new AtomicClock;
// ...
delete ptk;  // Undefined behavior! Only ~TimeKeeper called,
             // not ~AtomicClock. Memory leak!

// [GOOD] GOOD: Virtual destructor
class TimeKeeper {
public:
    TimeKeeper();
    virtual ~TimeKeeper();  // Virtual destructor
    // ...
};

TimeKeeper* ptk = new AtomicClock;
delete ptk;  // Correct! Calls ~AtomicClock, then ~TimeKeeper

// Complete example with virtual destructor:
class Shape {
public:
    virtual ~Shape() {
        std::cout << "~Shape()" << std::endl;
    }
    virtual void draw() const = 0;
};

class Circle : public Shape {
public:
    ~Circle() {
        std::cout << "~Circle()" << std::endl;
    }
    void draw() const override {
        std::cout << "Drawing circle" << std::endl;
    }
};

// Usage:
Shape* s = new Circle;
delete s;  // Output: ~Circle()
           //         ~Shape()
```

**When NOT to Declare Virtual Destructor**:

```cpp
// Class not intended as base class
class Point {
public:
    Point(int x, int y);
    ~Point();  // Non-virtual is fine

private:
    int x, y;
};

// Adding virtual destructor increases object size
// sizeof(Point) without virtual: 8 bytes (2 ints)
// sizeof(Point) with virtual: 16 bytes (2 ints + vptr)
```

**Pure Virtual Destructors**:

```cpp
class AWOV {  // Abstract w/o Virtuals
public:
    virtual ~AWOV() = 0;  // Pure virtual destructor
};

AWOV::~AWOV() { }  // Must provide definition!

// Usage:
class Derived : public AWOV {
    // ...
};
```

## 2.4 Item 8: Prevent exceptions from leaving destructors

**Concept**: Destructors should never emit exceptions. If they might, catch and handle them internally.

**Rationale**: If an exception leaves a destructor during stack unwinding (from another exception), it leads to undefined behavior or program termination.

**Example**:

```cpp
// [BAD] BAD: Exception can leave destructor
class DBConnection {
public:
    static DBConnection create();
    void close();  // May throw exception
};

class DBConn {
public:
    ~DBConn() {
        db.close();  // If this throws, program may terminate!
    }

private:
    DBConnection db;
};

// Problem scenario:
{
    DBConn dbc1;
    DBConn dbc2;
    // ...
}  // Destructors called
   // If dbc2.~DBConn() throws during unwinding from dbc1 exception,
   // program terminates or undefined behavior!
```

```cpp
// [GOOD] GOOD: Handle exceptions in destructor
class DBConn {
public:
    ~DBConn() {
        try {
            db.close();
        }
        catch (...) {
            // Log the error but don't propagate
            std::cerr << "Error closing database connection" << std::endl;
            std::abort();  // Or swallow the exception
        }
    }

private:
    DBConnection db;
};

// [GOOD] BETTER: Give clients opportunity to handle errors
class DBConn {
public:
    void close() {
        db.close();
        closed = true;
    }

    ~DBConn() {
        if (!closed) {
            try {
                db.close();
            }
            catch (...) {
                // Log error and swallow
                std::cerr << "Error in destructor" << std::endl;
            }
        }
    }

private:
    DBConnection db;
    bool closed = false;
};

// Usage:
{
    DBConn dbc;
    try {
        dbc.close();  // Give client chance to handle errors
    }
    catch (const std::exception& e) {
        // Handle error
    }
}  // If close() wasn't called, destructor handles it safely
```

## 2.5 Item 9: Never call virtual functions during construction or destruction

**Concept**: Virtual functions don't behave polymorphically during construction/destruction. The base class version is called instead of the derived class version.

**Example**:

```cpp
class Transaction {
public:
    Transaction() {
        logTransaction();  // Calls Transaction::logTransaction
    }                      // NOT derived class version!

    virtual void logTransaction() const {
        std::cout << "Transaction log" << std::endl;
    }
};

class BuyTransaction : public Transaction {
public:
    virtual void logTransaction() const {
        std::cout << "Buy transaction log" << std::endl;
    }
};

// Usage:
BuyTransaction b;  // Output: "Transaction log"
                   // NOT "Buy transaction log"!

// Why? During BuyTransaction construction:
// 1. Transaction constructor runs first
// 2. Object type is Transaction at this point
// 3. logTransaction() calls Transaction version
// 4. BuyTransaction members not initialized yet
```

**The Problem**:

```cpp
class Transaction {
public:
    Transaction() {
        logTransaction();
    }

    virtual void logTransaction() const = 0;  // Pure virtual
};

class BuyTransaction : public Transaction {
public:
    virtual void logTransaction() const {
        // Access BuyTransaction members
    }
};

BuyTransaction b;  // Undefined behavior!
                   // Calls pure virtual from base constructor
```

**Solution 1: Make function non-virtual**:

```cpp
class Transaction {
public:
    explicit Transaction(const std::string& logInfo) {
        logTransaction(logInfo);
    }

    void logTransaction(const std::string& logInfo) const {
        std::cout << logInfo << std::endl;
    }
};

class BuyTransaction : public Transaction {
public:
    BuyTransaction()
        : Transaction(createLogString()) {
    }

private:
    static std::string createLogString() {
        return "Buy transaction log";
    }
};
```

## 2.6 Item 10: Have assignment operators return a reference to *this

**Concept**: Follow the convention that assignment operators return a reference to their left-hand argument.

**Example**:

```cpp
class Widget {
public:
    Widget& operator=(const Widget& rhs) {
        // ...
        return *this;
    }

    // Also applies to other assignment operators:
    Widget& operator+=(const Widget& rhs) {
        // ...
        return *this;
    }

    Widget& operator=(int rhs) {
        // ...
        return *this;
    }
};

// Enables chaining:
Widget w1, w2, w3;
w1 = w2 = w3;  // Same as: w1 = (w2 = w3);

// Also:
w1 += w2 += w3;
```

## 2.7 Item 11: Handle assignment to self in operator=

**Concept**: Assignment operators must handle self-assignment safely and efficiently.

**Example**:

```cpp
class Bitmap { /* ... */ };

class Widget {
public:
    // ...

private:
    Bitmap* pb;
};

// [BAD] BAD: Unsafe for self-assignment
Widget& Widget::operator=(const Widget& rhs) {
    delete pb;           // Delete old bitmap
    pb = new Bitmap(*rhs.pb);  // If rhs == *this, we just deleted rhs.pb!
    return *this;
}

// Self-assignment scenario:
Widget w;
// ...
w = w;  // Disaster! Deletes and then tries to copy deleted object

// Also can happen indirectly:
a[i] = a[j];  // If i == j
*px = *py;    // If px and py point to same object

// [GOOD] GOOD: Identity test
Widget& Widget::operator=(const Widget& rhs) {
    if (this == &rhs) return *this;  // Identity test

    delete pb;
    pb = new Bitmap(*rhs.pb);
    return *this;
}

// [GOOD] BETTER: Exception-safe (and handles self-assignment)
Widget& Widget::operator=(const Widget& rhs) {
    Bitmap* pOrig = pb;           // Remember original
    pb = new Bitmap(*rhs.pb);     // Point to copy of rhs's bitmap
    delete pOrig;                 // Delete original
    return *this;
}
// If "new Bitmap" throws, pb unchanged
// Also handles self-assignment (less efficiently)

// [GOOD] BEST: Copy-and-swap idiom
class Widget {
public:
    void swap(Widget& rhs) {
        using std::swap;
        swap(pb, rhs.pb);
    }

    Widget& operator=(const Widget& rhs) {
        Widget temp(rhs);     // Make a copy
        swap(temp);           // Swap with copy
        return *this;
    }  // temp (with old data) destroyed

    // Alternative implementation:
```

```
    Widget& operator=(Widget rhs) {  // Pass by value (copy made)
        swap(rhs);                   // Swap with copy
        return *this;
    }

private:
    Bitmap* pb;
};
```

## 2.8 Item 12: Copy all parts of an object

**Concept**: When writing copy functions (copy constructor and copy assignment), make sure to copy all data members and call copy functions of base classes.

**Example**:

```
// [BAD] BAD: Forgetting to copy a member
class Customer {
public:
    Customer(const Customer& rhs)
        : name(rhs.name) {  // Forgot lastTransaction!
    }

    Customer& operator=(const Customer& rhs) {
        name = rhs.name;    // Forgot lastTransaction!
        return *this;
    }

private:
    std::string name;
    Date lastTransaction;  // Not copied!
};

// [GOOD] GOOD: Copy all members
class Customer {
public:
    Customer(const Customer& rhs)
        : name(rhs.name),
          lastTransaction(rhs.lastTransaction) {
    }

    Customer& operator=(const Customer& rhs) {
        name = rhs.name;
        lastTransaction = rhs.lastTransaction;
        return *this;
    }

private:
    std::string name;
    Date lastTransaction;
};

// [BAD] BAD: Forgetting base class parts
class PriorityCustomer : public Customer {
public:
    PriorityCustomer(const PriorityCustomer& rhs)
        : priority(rhs.priority) {  // Forgot to call base class copy constructor!
    }

    PriorityCustomer& operator=(const PriorityCustomer& rhs) {
```

```
        priority = rhs.priority;    // Forgot to call base class assignment!
        return *this;
    }

private:
    int priority;
};

// [GOOD] GOOD: Copy base class parts
class PriorityCustomer : public Customer {
public:
    PriorityCustomer(const PriorityCustomer& rhs)
        : Customer(rhs),            // Call base class copy constructor
          priority(rhs.priority) {
    }

    PriorityCustomer& operator=(const PriorityCustomer& rhs) {
        Customer::operator=(rhs);   // Assign base class parts
        priority = rhs.priority;
        return *this;
    }

private:
    int priority;
};
```

**Avoid Code Duplication Between Copy Functions**:

```
// [BAD] BAD: Don't have one copy function call the other
class Widget {
public:
    Widget(const Widget& rhs);
    Widget& operator=(const Widget& rhs) {
        // Don't do this!
        // Widget(rhs);  // Constructs new object, doesn't affect *this
    }
};

// [GOOD] GOOD: Create a private init function
class Widget {
public:
    Widget(const Widget& rhs) {
        init(rhs);
    }

    Widget& operator=(const Widget& rhs) {
        init(rhs);
        return *this;
    }

private:
    void init(const Widget& rhs) {
        // Common initialization code
    }
};
```

---

# 3 Chapter 3: Resource Management

## 3.1 Item 13: Use objects to manage resources

**Concept**: Use RAII (Resource Acquisition Is Initialization) to ensure resources are properly released.

**Example**:

```
class Investment { /* ... */ };

Investment* createInvestment();  // Factory function

// [BAD] BAD: Manual resource management
void f() {
    Investment* pInv = createInvestment();
    // ...
    delete pInv;  // What if we return early? Exception? Memory leak!
}

// [GOOD] GOOD: RAII with smart pointers
void f() {
    std::unique_ptr<Investment> pInv(createInvestment());
    // ...
}  // Automatically deleted

// Smart pointer examples:
void demonstrateSmartPointers() {
    // unique_ptr: Exclusive ownership
    std::unique_ptr<Investment> up1(createInvestment());
    // std::unique_ptr<Investment> up2(up1);  // Error! Can't copy
    std::unique_ptr<Investment> up2(std::move(up1));  // OK: Transfer ownership
    // up1 is now null

    // shared_ptr: Shared ownership
    std::shared_ptr<Investment> sp1(createInvestment());
    std::shared_ptr<Investment> sp2(sp1);  // OK: Both own the resource
    // Resource deleted when last shared_ptr destroyed
}
```

**RAII for Other Resources**:

```
class Lock {
public:
    explicit Lock(Mutex* pm) : mutexPtr(pm) {
        mutexPtr->lock();
    }

    ~Lock() {
        mutexPtr->unlock();
    }

private:
    Mutex* mutexPtr;
};

// Usage:
Mutex m;
{
    Lock ml(&m);  // Lock acquired
    // Critical section
}  // Lock automatically released
```

```cpp
// Modern C++: std::lock_guard, std::unique_lock
std::mutex mtx;
{
    std::lock_guard<std::mutex> lock(mtx);
    // Critical section
}  // Automatically unlocked
```

### 3.2 Item 14: Think carefully about copying behavior in resource-managing classes

**Concept**: When creating RAII classes, decide what copying should mean.

**Options**: 1. Prohibit copying 2. Reference-count the resource 3. Copy the underlying resource (deep copy) 4. Transfer ownership

**Example**:

```cpp
// Option 1: Prohibit copying
class Lock {
public:
    explicit Lock(Mutex* pm) : mutexPtr(pm) {
        mutexPtr->lock();
    }

    ~Lock() {
        mutexPtr->unlock();
    }

    Lock(const Lock&) = delete;
    Lock& operator=(const Lock&) = delete;

private:
    Mutex* mutexPtr;
};

// Option 2: Reference-count the resource
class Lock {
public:
    explicit Lock(Mutex* pm)
        : mutexPtr(pm, [](Mutex* p) { p->unlock(); }) {
        mutexPtr->lock();
    }

private:
    std::shared_ptr<Mutex> mutexPtr;  // Custom deleter unlocks
};

// Option 3: Deep copy
class Bitmap { /* ... */ };

class Image {
public:
    Image(const Image& rhs)
        : bitmap(new Bitmap(*rhs.bitmap)) {  // Deep copy
    }

    Image& operator=(const Image& rhs) {
        if (this != &rhs) {
            delete bitmap;
            bitmap = new Bitmap(*rhs.bitmap);
        }
        return *this;
```

```
    }

    ~Image() {
        delete bitmap;
    }

private:
    Bitmap* bitmap;
};

// Option 4: Transfer ownership (like unique_ptr)
class ResourceHolder {
public:
    ResourceHolder(ResourceHolder&& rhs) noexcept
        : resource(rhs.resource) {
        rhs.resource = nullptr;
    }

    ResourceHolder& operator=(ResourceHolder&& rhs) noexcept {
        if (this != &rhs) {
            delete resource;
            resource = rhs.resource;
            rhs.resource = nullptr;
        }
        return *this;
    }

private:
    Resource* resource;
};
```

### 3.3 Item 15: Provide access to raw resources in resource-managing classes

**Concept**: Smart pointers and RAII classes should provide access to the underlying raw resource when needed.

**Example**:

```
class Investment { /* ... */ };

std::shared_ptr<Investment> pInv(createInvestment());

// Explicit conversion via get()
int daysHeld(const Investment* pi);

int days = daysHeld(pInv.get());  // get() returns raw pointer

// Implicit conversion can be convenient but risky
class Font {
public:
    explicit Font(FontHandle fh) : f(fh) {}
    ~Font() { releaseFont(f); }

    FontHandle get() const { return f; }  // Explicit conversion

    operator FontHandle() const { return f; }  // Implicit conversion

private:
    FontHandle f;
};
```

```
// Usage with explicit conversion:
Font f(getFontHandle());
changeFontSize(f.get(), 20);

// Usage with implicit conversion:
Font f2(getFontHandle());
changeFontSize(f2, 20);  // Automatically converts to FontHandle

// Danger of implicit conversion:
Font f1(getFontHandle());
FontHandle fh = f1;  // Copy raw handle!
// When f1 is destroyed, fh becomes dangling!
```

**Best Practice**:

```
class Font {
public:
    explicit Font(FontHandle fh) : f(fh) {}
    ~Font() { releaseFont(f); }

    // Prefer explicit conversion
    FontHandle get() const { return f; }

    // Delete implicit conversion to prevent misuse
    operator FontHandle() const = delete;

private:
    FontHandle f;
};
```

### 3.4 Item 16: Use the same form in corresponding uses of new and delete

**Concept**: If you use new, use `delete`. If you use new`[]`, use `delete[]`.

**Example**:

```
// [BAD] BAD: Mismatched new/delete
std::string* stringPtr1 = new std::string;
std::string* stringPtr2 = new std::string[100];

delete stringPtr1;      // Correct
delete stringPtr2;      // Error! Should be delete[]
delete[] stringPtr1;    // Error! Should be delete

// [GOOD] GOOD: Matched pairs
std::string* sp1 = new std::string;
delete sp1;

std::string* sp2 = new std::string[100];
delete[] sp2;

// Typedef confusion:
typedef std::string AddressLines[4];

std::string* pal = new AddressLines;  // Like: new string[4]
delete pal;     // [BAD] Error! Should be delete[]
delete[] pal;   // [GOOD] Correct

// Avoid confusion with smart pointers and containers:
std::vector<std::string> addresses(100);  // Better!
std::unique_ptr<std::string[]> up(new std::string[100]);  // Or this
```

### 3.5 Item 17: Store newed objects in smart pointers in standalone statements

**Concept**: Create and store dynamically allocated objects in smart pointers in their own statement to avoid potential resource leaks.

**Example**:

```
int priority();
void processWidget(std::shared_ptr<Widget> pw, int priority);

// [BAD] BAD: Potential resource leak
processWidget(std::shared_ptr<Widget>(new Widget), priority());

// Problem: Compiler may execute operations in this order:
// 1. new Widget
// 2. priority()
// 3. std::shared_ptr constructor
// If priority() throws, the Widget from step 1 leaks!

// [GOOD] GOOD: Separate statement
std::shared_ptr<Widget> pw(new Widget);
processWidget(pw, priority());

// [GOOD] BETTER: Use make_shared (C++11)
processWidget(std::make_shared<Widget>(), priority());
// make_shared is exception-safe and more efficient
```

---

## 4 Chapter 4: Designs and Declarations

### 4.1 Item 18: Make interfaces easy to use correctly and hard to use incorrectly

**Concept**: Good interfaces are intuitive, prevent errors, and guide users toward correct usage.

**Example**:

```
// [BAD] BAD: Error-prone interface
class Date {
public:
    Date(int month, int day, int year);
};

Date d1(30, 3, 1995);   // Oops! March 30 or 30th month?
Date d2(3, 40, 1995);   // Invalid day!

// [GOOD] GOOD: Type-safe interface
struct Day {
    explicit Day(int d) : val(d) {}
    int val;
};

struct Month {
    explicit Month(int m) : val(m) {}
    int val;
};

struct Year {
    explicit Year(int y) : val(y) {}
    int val;
};
```

```
class Date {
public:
    Date(const Month& m, const Day& d, const Year& y);
};

Date d1(Month(3), Day(30), Year(1995));  // Clear and correct
// Date d2(30, 3, 1995);  // Error! Won't compile

// [GOOD] BETTER: Restrict values
class Month {
public:
    static Month Jan() { return Month(1); }
    static Month Feb() { return Month(2); }
    // ... more months

private:
    explicit Month(int m) : val(m) {}
    int val;
};

Date d(Month::Mar(), Day(30), Year(1995));
```

**Const Correctness**:

```
// Return const to prevent errors
const Rational operator*(const Rational& lhs, const Rational& rhs);

Rational a, b, c;
// (a * b) = c;  // Error! Can't assign to const
if (a * b = c) // Error! Typo caught at compile time

// Consistency with built-in types:
int x, y, z;
// if (x * y = z) // Error! Can't assign to rvalue
```

**Factory Functions Returning Smart Pointers**:

```
// Prevent resource leaks
std::shared_ptr<Investment> createInvestment() {
    return std::shared_ptr<Investment>(new Investment);
}

// Even better: Custom deleter
std::shared_ptr<Investment> createInvestment() {
    return std::shared_ptr<Investment>(
        new Stock,
        [](Investment* p) {
            logDeletion(p);
            delete p;
        }
    );
}
```

## 4.2 Item 19: Treat class design as type design

**Concept**: Designing a class is designing a type. Consider these questions:

1.
2.
3.
4.
5.

6.
7.
8.
9.
10.
11.
12.

**Example**:

```
class Rational {
public:
    // Creation/destruction
    Rational(int numerator = 0, int denominator = 1);
    ~Rational() = default;

    // Copy operations
    Rational(const Rational& rhs) = default;
    Rational& operator=(const Rational& rhs) = default;

    // Move operations
    Rational(Rational&& rhs) noexcept = default;
    Rational& operator=(Rational&& rhs) noexcept = default;

    // Type conversions
    explicit operator double() const {
        return static_cast<double>(n) / d;
    }

    // Operators
    Rational& operator+=(const Rational& rhs);

    // Accessors
    int numerator() const { return n; }
    int denominator() const { return d; }

private:
    int n, d;

    // Helper to maintain invariant
    void reduce();  // Simplify fraction

    // Invariant: d != 0, gcd(n,d) = 1
};

// Non-member operators for symmetry
const Rational operator+(const Rational& lhs, const Rational& rhs);
const Rational operator*(const Rational& lhs, const Rational& rhs);
```

## 4.3 Item 20: Prefer pass-by-reference-to-const to pass-by-value

**Concept**: Passing by reference-to-const is typically more efficient and avoids the slicing problem.

**Example**:

```cpp
class Person {
public:
    Person();
    virtual ~Person();
    // ...

private:
    std::string name;
    std::string address;
};

class Student : public Person {
public:
    Student();
    ~Student();
    // ...

private:
    std::string schoolName;
    std::string schoolAddress;
};

// [BAD] BAD: Pass by value
bool validateStudent(Student s) {  // Copy constructor called!
    // ...
    return true;
}
// Cost: 6 constructor calls + 6 destructor calls
// (Person copy ctor, 2 strings in Person,
//  Student copy ctor, 2 strings in Student,
//  plus 6 destructors)

// [GOOD] GOOD: Pass by reference-to-const
bool validateStudent(const Student& s) {  // No copying!
    // ...
    return true;
}
// Cost: 0 constructor calls, 0 destructor calls

// Slicing problem:
class Window {
public:
    virtual void display() const {
        std::cout << "Window::display()" << std::endl;
    }
};

class WindowWithScrollBars : public Window {
public:
    virtual void display() const override {
        std::cout << "WindowWithScrollBars::display()" << std::endl;
    }
};

// [BAD] BAD: Pass by value causes slicing
void printNameAndDisplay(Window w) {
    w.display();  // Always calls Window::display()!
}

WindowWithScrollBars wwsb;
```

```
printNameAndDisplay(wwsb);  // Sliced! Calls Window::display()

// [GOOD] GOOD: Pass by reference preserves polymorphism
void printNameAndDisplay(const Window& w) {
    w.display();  // Calls correct version polymorphically
}

printNameAndDisplay(wwsb);  // Calls WindowWithScrollBars::display()
```

**Exception: Built-in types and STL iterators**:

```
// Pass built-in types by value
void process(int value);           // Good
void setCoordinate(double x, double y);  // Good

// STL iterators and function objects (designed to be passed by value)
template<typename Iter>
void advance(Iter& it, int n);  // Iterator by reference is OK

// Small user-defined types: measure before assuming pass-by-value is better!
```

## 4.4 Item 21: Don't try to return a reference when you must return an object

**Concept**: Never return a reference to a local object, a heap-allocated object that the caller must delete, or a local static object when multiple return values are possible.

**Example**:

```
class Rational {
public:
    Rational(int numerator = 0, int denominator = 1);
    // ...
private:
    int n, d;
    friend const Rational operator*(const Rational& lhs, const Rational& rhs);
};

// [BAD] BAD: Reference to local object
const Rational& operator*(const Rational& lhs, const Rational& rhs) {
    Rational result(lhs.n * rhs.n, lhs.d * rhs.d);
    return result;  // Disaster! Returning reference to local object
}

// [BAD] BAD: Heap allocation
const Rational& operator*(const Rational& lhs, const Rational& rhs) {
    Rational* result = new Rational(lhs.n * rhs.n, lhs.d * rhs.d);
    return *result;  // Who calls delete?
}

// Usage:
Rational a(1, 2);
Rational b(3, 4);
Rational c = a * b;  // Memory leak! Can't delete the result

// [BAD] BAD: Static object
const Rational& operator*(const Rational& lhs, const Rational& rhs) {
    static Rational result;
    result = Rational(lhs.n * rhs.n, lhs.d * rhs.d);
    return result;  // Not thread-safe, fails with multiple comparisons
}
```

```cpp
// Problem:
if ((a * b) == (c * d)) {
    // Both calls return reference to same static object!
}

// [GOOD] GOOD: Return by value
const Rational operator*(const Rational& lhs, const Rational& rhs) {
    return Rational(lhs.n * rhs.n, lhs.d * rhs.d);
}
// Compilers can optimize using RVO (Return Value Optimization)
```

### 4.5 Item 22: Declare data members private

**Concept**: Data members should be private to provide encapsulation, flexibility, and fine-grained access control.

**Rationale**: 1. **Syntactic consistency**: All access through functions 2. **Fine-grained access control**: read-only, write-only, read-write 3. **Encapsulation**: Can change implementation without affecting clients 4. **Invariants**: Can enforce constraints

**Example**:

```cpp
// [BAD] BAD: Public data
class Point {
public:
    int x;
    int y;
};

// Clients access directly:
Point p;
p.x = 10;
// Can't change to polar coordinates without breaking clients!

// [GOOD] GOOD: Private data with accessors
class Point {
public:
    int getX() const { return x; }
    int getY() const { return y; }
    void setX(int newX) { x = newX; }
    void setY(int newY) { y = newY; }

private:
    int x;
    int y;
};

// Can change implementation later:
class Point {
public:
    double getX() const { return r * std::cos(theta); }
    double getY() const { return r * std::sin(theta); }
    void setX(double newX) {
        double oldY = getY();
        r = std::sqrt(newX * newX + oldY * oldY);
        theta = std::atan2(oldY, newX);
    }
    // ...

private:
    double r;      // radius
    double theta;  // angle
```

```
};

// Fine-grained access control:
class AccessLevels {
public:
    int getReadOnly() const { return readOnly; }
    void setReadWrite(int value) { readWrite = value; }
    int getReadWrite() const { return readWrite; }
    void setWriteOnly(int value) { writeOnly = value; }

private:
    int noAccess;    // No access
    int readOnly;    // Read-only access
    int readWrite;   // Read-write access
    int writeOnly;   // Write-only access
};
```

**Protected is Not More Encapsulated Than Public**:

```
// If protected data breaks, all derived classes may break
// If public data breaks, all clients may break
// Both have large dependent code bases!

// Prefer private data with protected accessors if needed
class Base {
public:
    // ...

protected:
    int getValue() const { return value; }
    void setValue(int v) { value = v; }

private:
    int value;  // Can change implementation!
};
```

## 4.6 Item 23: Prefer non-member non-friend functions to member functions

**Concept**: Non-member functions can increase encapsulation by reducing the number of functions that can access private data.

**Example**:

```
class WebBrowser {
public:
    void clearCache();
    void clearHistory();
    void removeCookies();

    // [BAD] Option 1: Member function
    void clearEverything() {
        clearCache();
        clearHistory();
        removeCookies();
    }
};

// [GOOD] Option 2: Non-member function (better encapsulation!)
void clearBrowser(WebBrowser& wb) {
    wb.clearCache();
    wb.clearHistory();
```

```
    wb.removeCookies();
}

// Why is non-member better?
// - Member function: 4 functions can access private data
// - Non-member: 3 functions can access private data
// - More encapsulation!
```

**Namespace Organization (C++ idiom)**:

```
// webbrowser.h - class definition
namespace WebBrowserStuff {
    class WebBrowser { /* ... */ };
    // Core functionality
}

// webbrowserbookmarks.h - bookmark conveniences
namespace WebBrowserStuff {
    void addBookmark(WebBrowser& wb, const std::string& url);
    void removeBookmark(WebBrowser& wb, const std::string& url);
    // ...
}

// webbrowsercookies.h - cookie conveniences
namespace WebBrowserStuff {
    void validateCookies(const WebBrowser& wb);
    // ...
}

// Clients include only what they need:
#include "webbrowser.h"
#include "webbrowserbookmarks.h"  // Only if needed
```

## 4.7 Item 24: Declare non-member functions when type conversions should apply to all parameters

**Concept**: For operators that should support implicit conversions on all operands, make them non-member functions.

**Example**:

```
class Rational {
public:
    Rational(int numerator = 0, int denominator = 1);  // Implicit conversion!
    int numerator() const;
    int denominator() const;

    // [BAD] BAD: Member function
    const Rational operator*(const Rational& rhs) const {
        return Rational(numerator() * rhs.numerator(),
                        denominator() * rhs.denominator());
    }
};

// Usage:
Rational oneHalf(1, 2);
Rational result;

result = oneHalf * 2;  // OK: oneHalf.operator*(2)
                       // 2 implicitly converted to Rational(2, 1)
```

```
result = 2 * oneHalf;  // Error! No operator* for int
                       // Would need: 2.operator*(oneHalf)

// [GOOD] GOOD: Non-member function
class Rational {
public:
    Rational(int numerator = 0, int denominator = 1);
    int numerator() const;
    int denominator() const;
};

const Rational operator*(const Rational& lhs, const Rational& rhs) {
    return Rational(lhs.numerator() * rhs.numerator(),
                    lhs.denominator() * rhs.denominator());
}

// Now both work:
result = oneHalf * 2;  // operator*(oneHalf, Rational(2))
result = 2 * oneHalf;  // operator*(Rational(2), oneHalf)
```

**Should operator* be a friend?**

```
// Usually NO! Use public interface:
const Rational operator*(const Rational& lhs, const Rational& rhs) {
    return Rational(lhs.numerator() * rhs.numerator(),
                    lhs.denominator() * rhs.denominator());
}
// No friend declaration needed!

// Only make it friend if it must access private data directly
```

## 4.8 Item 25: Consider support for a non-throwing swap

**Concept**: The default `std::swap` can be inefficient for some types. Provide a specialized swap when appropriate.

**Example**:

```
// Default swap (inefficient for some types):
template<typename T>
void swap(T& a, T& b) {
    T temp(a);  // Copy construct
    a = b;      // Copy assign
    b = temp;   // Copy assign
}

// Problem with pimpl idiom:
class WidgetImpl {
public:
    // ...
private:
    int a, b, c;
    std::vector<double> v;  // Lots of data
};

class Widget {
public:
    Widget(const Widget& rhs);
    Widget& operator=(const Widget& rhs) {
        *pImpl = *(rhs.pImpl);  // Copy data
        return *this;
```

```
    }
    // ...

private:
    WidgetImpl* pImpl;  // Pointer to implementation
};

// Default swap copies all data three times!
Widget w1, w2;
std::swap(w1, w2);  // Inefficient!

// [GOOD] SOLUTION 1: Provide public swap member function
class Widget {
public:
    void swap(Widget& other) {
        using std::swap;
        swap(pImpl, other.pImpl);  // Just swap pointers!
    }
    // ...
};

// [GOOD] SOLUTION 2: Specialize std::swap
namespace std {
    template<>
    void swap<Widget>(Widget& a, Widget& b) {
        a.swap(b);  // Call member swap
    }
}

// Usage:
Widget w1, w2;
std::swap(w1, w2);  // Now efficient!
```

**For Class Templates**:

```
template<typename T>
class WidgetImpl { /* ... */ };

template<typename T>
class Widget {
public:
    void swap(Widget& other) {
        using std::swap;
        swap(pImpl, other.pImpl);
    }
    // ...

private:
    WidgetImpl<T>* pImpl;
};

// Can't partially specialize function templates in std
// Solution: Use non-member function in same namespace
namespace WidgetStuff {
    template<typename T>
    class Widget { /* ... */ };

    template<typename T>
    void swap(Widget<T>& a, Widget<T>& b) {  // Non-member swap
        a.swap(b);
    }
```

```
}

// Usage with ADL (Argument-Dependent Lookup):
template<typename T>
void doSomething(T& obj1, T& obj2) {
    using std::swap;  // Make std::swap available
    swap(obj1, obj2); // Calls best swap via ADL
}
```

---

# 5 Chapter 5: Implementations

## 5.1 Item 26: Postpone variable definitions as long as possible

**Concept**: Define variables when you have initialization values, not before.

**Example**:

```
// [BAD] BAD: Premature definition
std::string encryptPassword(const std::string& password) {
    std::string encrypted;  // Default constructed too early!

    if (password.length() < 8) {
        throw std::logic_error("Password too short");
    }

    encrypted = password;  // Assignment (could have been initialization)
    encrypt(encrypted);
    return encrypted;
}
// If exception thrown, encrypted constructed for nothing!

// [GOOD] GOOD: Postpone until you have initialization value
std::string encryptPassword(const std::string& password) {
    if (password.length() < 8) {
        throw std::logic_error("Password too short");
    }

    std::string encrypted(password);  // Direct initialization!
    encrypt(encrypted);
    return encrypted;
}
```

**Loops**:

```cpp
// Approach A: Define outside loop
Widget w;
for (int i = 0; i < n; ++i) {
    w = /* some value */;
    // ...
}
// Cost: 1 constructor + 1 destructor + n assignments

// Approach B: Define inside loop
for (int i = 0; i < n; ++i) {
    Widget w(/* some value */);
    // ...
}
// Cost: n constructors + n destructors

// Prefer A if: assignment less expensive than ctor+dtor
//              and you're performance-critical
// Prefer B otherwise (better encapsulation, smaller scope)
```

## 5.2 Item 27: Minimize casting

**Concept**: Casts are dangerous. Avoid them when possible; use C++-style casts when necessary.

**C++ Style Casts**:

```cpp
// C-style cast (avoid!):
(T) expression
T(expression)

// C++ style casts (preferred):
const_cast<T>(expression)        // Remove const/volatile
static_cast<T>(expression)       // Compile-time conversions
dynamic_cast<T>(expression)      // Safe downcasting
reinterpret_cast<T>(expression)  // Low-level reinterpretation

// Examples:
class Widget { /* ... */ };

const Widget w;
Widget* pw = const_cast<Widget*>(&w);  // Remove const

double d = 3.14;
int i = static_cast<int>(d);  // Explicit conversion

class Base { public: virtual ~Base() {} };
class Derived : public Base { /* ... */ };

Base* pb = new Derived;
Derived* pd = dynamic_cast<Derived*>(pb);  // Safe downcast

int* pi = reinterpret_cast<int*>(pb);  // Dangerous!
```

**Casts Can Create Temporary Objects**:

```cpp
class Base {
public:
    virtual void doSomething() {
        std::cout << "Base::doSomething" << std::endl;
    }
};

class Derived : public Base {
public:
    virtual void doSomething() override {
        static_cast<Base>(*this).doSomething();  // [BAD] Wrong!
        // Calls Base::doSomething on COPY of *this!
        std::cout << "Derived::doSomething" << std::endl;
    }
};

// [GOOD] Correct approach:
class Derived : public Base {
public:
    virtual void doSomething() override {
        Base::doSomething();  // Call directly
        std::cout << "Derived::doSomething" << std::endl;
    }
};
```

**Dynamic Cast Performance**:

```cpp
class Window { public: virtual ~Window() {} };
class SpecialWindow : public Window {
public:
    void blink();
};

typedef std::vector<std::shared_ptr<Window>> VPW;
VPW winPtrs;

// [BAD] BAD: dynamic_cast in loop
for (VPW::iterator it = winPtrs.begin(); it != winPtrs.end(); ++it) {
    if (SpecialWindow* psw = dynamic_cast<SpecialWindow*>(it->get())) {
        psw->blink();
    }
}

// [GOOD] BETTER: Store derived type directly
typedef std::vector<std::shared_ptr<SpecialWindow>> VPSW;
VPSW specialWinPtrs;

for (VPSW::iterator it = specialWinPtrs.begin();
     it != specialWinPtrs.end(); ++it) {
    (*it)->blink();  // No cast needed
}

// [GOOD] ALTERNATIVE: Virtual function
class Window {
public:
    virtual void blink() {}  // Default: do nothing
    virtual ~Window() {}
};

class SpecialWindow : public Window {
public:
```

```
    virtual void blink() override { /* ... */ }
};

for (VPW::iterator it = winPtrs.begin(); it != winPtrs.end(); ++it) {
    (*it)->blink();  // Polymorphism, no cast
}
```

## 5.3 Item 28: Avoid returning "handles" to object internals

**Concept**: Don't return references, pointers, or iterators to internal data unless you want to expose implementation details and risk dangling handles.

**Example**:

```
struct RectData {
    Point ulhc;  // Upper-left hand corner
    Point lrhc;  // Lower-right hand corner
};

class Rectangle {
public:
    // [BAD] BAD: Returns reference to internal data
    Point& upperLeft() { return pData->ulhc; }
    Point& lowerRight() { return pData->lrhc; }

private:
    std::shared_ptr<RectData> pData;
};

// Problem 1: Violates encapsulation
Rectangle rect;
rect.upperLeft().setX(50);  // Modifies internal state!

// Problem 2: Const member function isn't const
const Rectangle rect2;
rect2.upperLeft().setX(50);  // Modifies "const" object!

// [GOOD] BETTER: Return const reference
class Rectangle {
public:
    const Point& upperLeft() const { return pData->ulhc; }
    const Point& lowerRight() const { return pData->lrhc; }

private:
    std::shared_ptr<RectData> pData;
};

// Still has problem: Dangling references
const Point& getUpperLeft(const Rectangle& rect) {
    return rect.upperLeft();
}

Rectangle createRect();  // Factory function

const Point& p = getUpperLeft(createRect());
// Temporary Rectangle destroyed!
// p is now dangling!

// [GOOD] BEST: Return by value
class Rectangle {
```

```
public:
    Point upperLeft() const { return pData->ulhc; }
    Point lowerRight() const { return pData->lrhc; }

private:
    std::shared_ptr<RectData> pData;
};
```

## 5.4 Item 29: Strive for exception-safe code

**Concept**: Exception-safe functions guarantee one of three levels: 1. **Basic guarantee**: Invariants preserved, no resource leaks 2. **Strong guarantee**: All-or-nothing (rollback on failure) 3. **No-throw guarantee**: Never throw exceptions

**Example**:

```
class PrettyMenu {
public:
    void changeBackground(std::istream& imgSrc);

private:
    Mutex mutex;
    Image* bgImage;
    int imageChanges;
};

// [BAD] BAD: Not exception-safe
void PrettyMenu::changeBackground(std::istream& imgSrc) {
    lock(&mutex);
    delete bgImage;
    ++imageChanges;
    bgImage = new Image(imgSrc);  // If this throws:
                                  // 1. Mutex never unlocked
                                  // 2. bgImage points to deleted object
                                  // 3. imageChanges incremented incorrectly
    unlock(&mutex);
}

// [GOOD] BETTER: Basic guarantee with RAII
void PrettyMenu::changeBackground(std::istream& imgSrc) {
    Lock ml(&mutex);  // RAII for mutex

    delete bgImage;
    ++imageChanges;
    bgImage = new Image(imgSrc);
}  // Mutex automatically unlocked
// Still problems: bgImage and imageChanges inconsistent if exception

// [GOOD] GOOD: Strong guarantee
class PrettyMenu {
public:
    void changeBackground(std::istream& imgSrc);

private:
    Mutex mutex;
    std::shared_ptr<Image> bgImage;  // Smart pointer
    int imageChanges;
};

void PrettyMenu::changeBackground(std::istream& imgSrc) {
```

```
    Lock ml(&mutex);

    bgImage.reset(new Image(imgSrc));  // Replace if no exception
    ++imageChanges;
}

// [GOOD] BETTER: Copy and swap
class PrettyMenu {
public:
    void changeBackground(std::istream& imgSrc);

private:
    struct PMImpl {
        std::shared_ptr<Image> bgImage;
        int imageChanges;
    };

    Mutex mutex;
    std::shared_ptr<PMImpl> pImpl;
};

void PrettyMenu::changeBackground(std::istream& imgSrc) {
    using std::swap;

    Lock ml(&mutex);

    std::shared_ptr<PMImpl> pNew(new PMImpl(*pImpl));  // Copy
    pNew->bgImage.reset(new Image(imgSrc));  // Modify copy
    ++pNew->imageChanges;

    swap(pImpl, pNew);  // Atomic swap
}  // Old state destroyed
```

**No-throw guarantee**:

```
// Functions that should never throw:
// - Destructors
// - swap functions
// - Move operations (when possible)

class Widget {
public:
    void swap(Widget& other) noexcept {
        using std::swap;
        swap(pImpl, other.pImpl);
    }

    Widget(Widget&& rhs) noexcept
        : pImpl(std::move(rhs.pImpl)) {}

private:
    std::shared_ptr<WidgetImpl> pImpl;
};
```

## 5.5 Item 30: Understand the ins and outs of inlining

**Concept**: Inline functions can improve performance by eliminating function call overhead, but they can also increase code size.

**When to inline**: - Small, frequently called functions - Functions in headers that need high performance - Template functions (often must be in headers)

**When NOT to inline**: - Large functions (code bloat) - Recursive functions - Virtual functions (can't inline through pointer/reference) - Functions called through function pointers

**Example**:

```
// Implicit inline (defined in class body):
class Person {
public:
    int age() const { return theAge; }  // Implicitly inline

private:
    int theAge;
};

// Explicit inline:
class Person {
public:
    int age() const;  // Declaration

private:
    int theAge;
};

inline int Person::age() const {  // Definition
    return theAge;
}

// [BAD] BAD: Inlining large function
inline void bigFunction() {
    // 100 lines of code
}
// Every call replaces with 100 lines!

// Virtual functions can't be inlined (usually):
class Base {
public:
    virtual void func() const {}
};

inline void Base::func() const { }  // Meaningless for virtual calls

Base* pb = new Derived;
pb->func();  // Can't inline (don't know type at compile time)

Base b;
b.func();  // Could inline (type known)

// Constructors and destructors are poor inline candidates:
class Derived : public Base {
public:
    Derived() {  // Looks simple, but...
        // Compiler adds:
        // - Base class constructor call
        // - Member initialization
        // - Exception handling code
    }

private:
    std::string s1, s2;
    std::vector<int> v;
};
```

```
// Actual code is much larger than it appears!
```

## 5.6 Item 31: Minimize compilation dependencies between files

**Concept**: Reduce compilation dependencies by using declarations instead of definitions, and by using the Pimpl idiom.

**Problem**:

```
// person.h
#include <string>
#include "date.h"
#include "address.h"

class Person {
public:
    Person(const std::string& name, const Date& birthday,
           const Address& addr);
    std::string name() const;
    // ...

private:
    std::string theName;     // Requires <string>
    Date theBirthDate;       // Requires "date.h"
    Address theAddress;      // Requires "address.h"
};

// Problem: Any change to Date or Address forces recompilation
// of Person and all files that include person.h!
```

**Solution 1: Forward Declarations**:

```
// person.h
#include <string>
#include <memory>

class Date;     // Forward declaration
class Address;  // Forward declaration

class Person {
public:
    Person(const std::string& name, const Date& birthday,
           const Address& addr);
    std::string name() const;
    // ...

private:
    std::string theName;
    std::shared_ptr<Date> theBirthDate;
    std::shared_ptr<Address> theAddress;
};
```

**Solution 2: Pimpl (Pointer to Implementation) Idiom**:

```cpp
// person.h - Interface
#include <string>
#include <memory>

class PersonImpl;  // Forward declaration
class Date;
class Address;

class Person {
public:
    Person(const std::string& name, const Date& birthday,
           const Address& addr);
    std::string name() const;
    // ...

private:
    std::shared_ptr<PersonImpl> pImpl;  // Pointer to implementation
};

// person.cpp - Implementation
#include "person.h"
#include "personimpl.h"  // Contains PersonImpl definition

Person::Person(const std::string& name, const Date& birthday,
               const Address& addr)
    : pImpl(std::make_shared<PersonImpl>(name, birthday, addr)) {
}

std::string Person::name() const {
    return pImpl->name();
}

// personimpl.h
#include <string>
#include "date.h"
#include "address.h"

class PersonImpl {
public:
    PersonImpl(const std::string& name, const Date& birthday,
               const Address& addr);
    std::string name() const { return theName; }
    // ...

private:
    std::string theName;
    Date theBirthDate;
    Address theAddress;
};
```

**Interface Classes (Abstract Base Classes)**:

```
// person.h
class Person {
public:
    virtual ~Person();
    virtual std::string name() const = 0;
    virtual std::string birthDate() const = 0;
    // ...

    static std::shared_ptr<Person> create(const std::string& name,
                                          const Date& birthday,
                                          const Address& addr);
};

// person.cpp
class RealPerson : public Person {
public:
    RealPerson(const std::string& name, const Date& birthday,
               const Address& addr)
        : theName(name), theBirthDate(birthday), theAddress(addr) {}

    virtual ~RealPerson() {}

    std::string name() const override { return theName; }
    std::string birthDate() const override { /* ... */ }

private:
    std::string theName;
    Date theBirthDate;
    Address theAddress;
};

std::shared_ptr<Person> Person::create(const std::string& name,
                                       const Date& birthday,
                                       const Address& addr) {
    return std::make_shared<RealPerson>(name, birthday, addr);
}

// Client usage:
std::shared_ptr<Person> pp = Person::create(name, dateOfBirth, address);
std::cout << pp->name();
```

---

# 6 Chapter 6: Inheritance and Object-Oriented Design

## 6.1 Item 32: Make sure public inheritance models “is-a”

**Concept**: Public inheritance means “is-a”. Every derived class object IS-A base class object.

**Example**:

```cpp
// If D publicly inherits from B, then:
// - Every D is a B
// - Everything true of B must be true of D
// - Anywhere B can be used, D can be used

class Person { /* ... */ };
class Student : public Person { /* ... */ };

void eat(const Person& p);  // Anyone can eat
void study(const Student& s);  // Only students study

Person p;
Student s;

eat(p);   // OK: p is a Person
eat(s);   // OK: s is a Student, and every Student is a Person

study(s); // OK: s is a Student
// study(p); // Error: not every Person is a Student
```

**Be Careful of Counter-intuitive Relationships**:

```cpp
// [BAD] BAD: Penguins are birds, but they can't fly!
class Bird {
public:
    virtual void fly() {
        std::cout << "Flying" << std::endl;
    }
};

class Penguin : public Bird {
    // What to do here? Penguins can't fly!
};

// [GOOD] SOLUTION 1: Separate flying birds
class Bird {
    // No fly function
};

class FlyingBird : public Bird {
public:
    virtual void fly() { /* ... */ }
};

class Penguin : public Bird {
    // No fly function
};

class Eagle : public FlyingBird {
    // Can fly
};

// [GOOD] SOLUTION 2: Runtime error
class Penguin : public Bird {
public:
    virtual void fly() override {
        throw std::logic_error("Penguins can't fly!");
    }
};

// [GOOD] SOLUTION 3: Compile-time error (better!)
```

```cpp
class Bird {
    // No fly declared
};

class Penguin : public Bird {
    // No fly - compile error if you try to call it
};
```

**Squares and Rectangles**:

```cpp
// [BAD] BAD: Mathematical is-a doesn't always work in code
class Rectangle {
public:
    virtual void setHeight(int h) { height = h; }
    virtual void setWidth(int w) { width = w; }
    virtual int getHeight() const { return height; }
    virtual int getWidth() const { return width; }

private:
    int height, width;
};

class Square : public Rectangle {
public:
    virtual void setHeight(int h) override {
        height = h;
        width = h;  // Maintain square invariant
    }

    virtual void setWidth(int w) override {
        height = w;
        width = w;
    }
};

// Problem:
void makeBigger(Rectangle& r) {
    int oldHeight = r.getHeight();
    r.setWidth(r.getWidth() + 10);
    assert(r.getHeight() == oldHeight);  // Should be true for rectangles!
}

Square s;
makeBigger(s);  // Assertion fails! Square is NOT a Rectangle!
```

## 6.2 Item 33: Avoid hiding inherited names

**Concept**: Names in derived classes hide names in base classes, even if they have different parameter types.

**Example**:

```cpp
class Base {
public:
    virtual void mf1() = 0;
    virtual void mf1(int);

    virtual void mf2();

    void mf3();
    void mf3(double);

private:
    int x;
};

class Derived : public Base {
public:
    virtual void mf1();  // Hides Base::mf1(int)
    void mf3();          // Hides both Base::mf3() and Base::mf3(double)
    void mf4();
};

// Usage:
Derived d;
int x;

d.mf1();     // OK: calls Derived::mf1
// d.mf1(x);    // Error! Derived::mf1 hides Base::mf1(int)

d.mf2();     // OK: calls Base::mf2

d.mf3();     // OK: calls Derived::mf3
// d.mf3(x);    // Error! Derived::mf3 hides Base::mf3(double)

// [GOOD] SOLUTION: using declarations
class Derived : public Base {
public:
    using Base::mf1;  // Make all Base::mf1 visible
    using Base::mf3;  // Make all Base::mf3 visible

    virtual void mf1();
    void mf3();
};

// Now works:
Derived d;
int x;
d.mf1();    // Calls Derived::mf1
d.mf1(x);   // Calls Base::mf1(int)
d.mf3();    // Calls Derived::mf3
d.mf3(x);   // Calls Base::mf3(double)
```

**Private Inheritance and Forwarding**:

```cpp
class Base {
public:
    virtual void mf1() = 0;
    virtual void mf1(int);
};

class Derived : private Base {  // Private inheritance
public:
    // Don't want all versions, just forward specific one:
    virtual void mf1() override {
        Base::mf1();
    }
};

Derived d;
// d.mf1(5);  // Error: Base::mf1(int) not accessible
d.mf1();      // OK: calls forwarding function
```

## 6.3 Item 34: Differentiate between inheritance of interface and inheritance of implementation

**Concept**: Understand the differences between pure virtual, simple virtual, and non-virtual functions.

**Three Types of Functions**: 1. **Pure virtual**: Inherit interface only (must override) 2. **Simple virtual**: Inherit interface + default implementation (may override) 3. **Non-virtual**: Inherit interface + mandatory implementation (invariant)

**Example**:

```cpp
class Shape {
public:
    // Pure virtual: Derived classes MUST provide implementation
    virtual void draw() const = 0;

    // Simple virtual: Default implementation provided, can override
    virtual void error(const std::string& msg) const {
        std::cerr << "Shape error: " << msg << std::endl;
    }

    // Non-virtual: Invariant over specialization
    int objectID() const {
        return id;
    }

private:
    int id;
};

class Rectangle : public Shape {
public:
    virtual void draw() const override {  // Must implement
        // Draw rectangle
    }

    // Inherits error(), can override if needed
};

class Ellipse : public Shape {
public:
    virtual void draw() const override {  // Must implement
        // Draw ellipse
```

```
    }

    virtual void error(const std::string& msg) const override {
        std::cerr << "Ellipse error: " << msg << std::endl;
    }
};
```

**Danger of Default Implementation in Virtual Functions**:

```
class Airport { /* ... */ };

class Airplane {
public:
    virtual void fly(const Airport& destination) {
        // Default code for flying to destination
        std::cout << "Flying normally" << std::endl;
    }
};

class ModelA : public Airplane { /* ... */ };  // Uses default fly
class ModelB : public Airplane { /* ... */ };  // Uses default fly

// New plane added:
class ModelC : public Airplane {
    // Forgot to override fly()!
    // Uses default, but ModelC flies differently!
};

// [GOOD] SOLUTION 1: Pure virtual with protected default
class Airplane {
public:
    virtual void fly(const Airport& destination) = 0;  // Must override

protected:
    void defaultFly(const Airport& destination) {
        // Default implementation
    }
};

class ModelA : public Airplane {
public:
    virtual void fly(const Airport& destination) override {
        defaultFly(destination);  // Use default
    }
};

class ModelC : public Airplane {
public:
    virtual void fly(const Airport& destination) override {
        // Custom implementation for ModelC
    }
};

// [GOOD] SOLUTION 2: Pure virtual with default implementation
class Airplane {
public:
    virtual void fly(const Airport& destination) = 0;
};

void Airplane::fly(const Airport& destination) {
    // Default implementation
```

```
}

class ModelA : public Airplane {
public:
    virtual void fly(const Airport& destination) override {
        Airplane::fly(destination);  // Explicitly call default
    }
};
```

## 6.4 Item 35: Consider alternatives to virtual functions

**Concept**: Virtual functions are not the only way to implement polymorphic behavior.

**Alternative 1: Template Method Pattern via NVI (Non-Virtual Interface)**:

```
class GameCharacter {
public:
    int healthValue() const {  // Non-virtual public interface
        // Before stuff
        int retVal = doHealthValue();  // Virtual implementation
        // After stuff
        return retVal;
    }

private:
    virtual int doHealthValue() const {  // Virtual implementation
        // Default calculation
        return 100;
    }
};

// Advantage: Can do before/after work around virtual call
```

**Alternative 2: Strategy Pattern via Function Pointers**:

```
class GameCharacter;  // Forward declaration

int defaultHealthCalc(const GameCharacter& gc);

class GameCharacter {
public:
    typedef int (*HealthCalcFunc)(const GameCharacter&);

    explicit GameCharacter(HealthCalcFunc hcf = defaultHealthCalc)
        : healthFunc(hcf) {}

    int healthValue() const {
        return healthFunc(*this);
    }

private:
    HealthCalcFunc healthFunc;
};

// Different health calculation functions:
int loseHealthQuickly(const GameCharacter& gc) { return 50; }
int loseHealthSlowly(const GameCharacter& gc) { return 90; }

// Usage:
GameCharacter warrior(loseHealthQuickly);
GameCharacter healer(loseHealthSlowly);
```

```
// Can even change strategy at runtime:
// warrior.setHealthCalculator(loseHealthSlowly);
```

**Alternative 3: Strategy Pattern via std::function**:

```
class GameCharacter;

class GameCharacter {
public:
    typedef std::function<int (const GameCharacter&)> HealthCalcFunc;

    explicit GameCharacter(HealthCalcFunc hcf = defaultHealthCalc)
        : healthFunc(hcf) {}

    int healthValue() const {
        return healthFunc(*this);
    }

private:
    HealthCalcFunc healthFunc;
};

// Now can use:
// - Function objects
// - Lambda expressions
// - Member functions
short calcHealth(const GameCharacter&);  // Different return type OK

class HealthCalculator {
public:
    int operator()(const GameCharacter&) const { return 80; }
};

GameCharacter gc1(calcHealth);
GameCharacter gc2(HealthCalculator());
GameCharacter gc3([](const GameCharacter&) { return 75; });
```

**Alternative 4: Classic Strategy Pattern**:

```
class GameCharacter;

class HealthCalcFunc {
public:
    virtual ~HealthCalcFunc() {}
    virtual int calc(const GameCharacter& gc) const = 0;
};

HealthCalcFunc defaultHealthCalc;

class GameCharacter {
public:
    explicit GameCharacter(HealthCalcFunc* phcf = &defaultHealthCalc)
        : pHealthCalc(phcf) {}

    int healthValue() const {
        return pHealthCalc->calc(*this);
    }

private:
    HealthCalcFunc* pHealthCalc;
};
```

## 6.5 Item 36: Never redefine an inherited non-virtual function

**Concept**: Non-virtual functions are statically bound. Redefining them leads to confusion.

**Example**:

```
class B {
public:
    void mf() {
        std::cout << "B::mf()" << std::endl;
    }
};

class D : public B {
public:
    void mf() {  // [BAD] Hides B::mf()
        std::cout << "D::mf()" << std::endl;
    }
};

D x;
B* pB = &x;
D* pD = &x;

pB->mf();  // Calls B::mf()!
pD->mf();  // Calls D::mf()!

// Same object, different behavior based on pointer type!
// This violates public inheritance "is-a" relationship
```

**Why This is Wrong**:

```
// If D is-a B, and B has a non-virtual function mf,
// then D should have exactly that same behavior.
//
// If D needs different behavior, mf should be virtual in B.
// If D can't/shouldn't change behavior, don't redefine mf in D.
```

## 6.6 Item 37: Never redefine a function's inherited default parameter value

**Concept**: Default parameters are statically bound, but virtual functions are dynamically bound. This creates confusion.

**Example**:

```
class Shape {
public:
    enum ShapeColor { Red, Green, Blue };

    virtual void draw(ShapeColor color = Red) const = 0;
};

class Rectangle : public Shape {
public:
    // [BAD] BAD: Different default parameter
    virtual void draw(ShapeColor color = Green) const override {
        std::cout << "Rectangle::draw(" << color << ")" << std::endl;
    }
};

class Circle : public Shape {
public:
    // [GOOD] GOOD: Same default (but still not ideal)
```

```
    virtual void draw(ShapeColor color = Red) const override {
        std::cout << "Circle::draw(" << color << ")" << std::endl;
    }
};

// Problem:
Shape* ps;
Shape* pc = new Circle;
Shape* pr = new Rectangle;

pc->draw();  // Calls Circle::draw(Red) - OK
pr->draw();  // Calls Rectangle::draw(Red) - Used base class default!
             // Not Rectangle::draw(Green)

// Default parameters are statically bound!

// [GOOD] SOLUTION: Use NVI idiom
class Shape {
public:
    enum ShapeColor { Red, Green, Blue };

    void draw(ShapeColor color = Red) const {  // Non-virtual
        doDraw(color);
    }

private:
    virtual void doDraw(ShapeColor color) const = 0;  // No default param
};

class Rectangle : public Shape {
private:
    virtual void doDraw(ShapeColor color) const override {
        std::cout << "Rectangle::draw(" << color << ")" << std::endl;
    }
};
```

## 6.7 Item 38: Model “has-a” or “is-implemented-in-terms-of” through composition

**Concept**: Composition (having an object as a member) models either “has-a” (application domain) or “is-implemented-in-terms-of” (implementation domain).

**Example - Has-A (Application Domain)**:

```
class Address { /* ... */ };
class PhoneNumber { /* ... */ };

class Person {
public:
    // ...

private:
    std::string name;          // Person has-a name
    Address address;           // Person has-a address
    PhoneNumber voiceNumber;   // Person has-a phone number
    PhoneNumber faxNumber;
};
```

**Example - Implemented-In-Terms-Of (Implementation Domain)**:

```cpp
// Want a Set implemented using std::list
// Set is-NOT-a list (can have duplicates)
// Set is-implemented-in-terms-of list

// [BAD] WRONG: Public inheritance
template<typename T>
class Set : public std::list<T> { /* ... */ };
// This says Set is-a list, which is wrong!

// [GOOD] CORRECT: Composition
template<typename T>
class Set {
public:
    bool member(const T& item) const {
        return std::find(rep.begin(), rep.end(), item) != rep.end();
    }

    void insert(const T& item) {
        if (!member(item)) {
            rep.push_back(item);
        }
    }

    void remove(const T& item) {
        auto it = std::find(rep.begin(), rep.end(), item);
        if (it != rep.end()) {
            rep.erase(it);
        }
    }

    std::size_t size() const {
        return rep.size();
    }

private:
    std::list<T> rep;  // Representation: Set implemented using list
};
```

## 6.8 Item 39: Use private inheritance judiciously

**Concept**: Private inheritance means “is-implemented-in-terms-of”. It’s an implementation technique, not a design relationship.

**Differences from Composition**: 1. Compilers don’t convert derived to base (private inheritance) 2. Members inherited from private base become private in derived 3. Can access protected members and override virtual functions

**Example**:

```cpp
class Timer {
public:
    explicit Timer(int tickFrequency);
    virtual void onTick() const;  // Called on each tick
};

// [BAD] Option 1: Public inheritance (wrong!)
class Widget : public Timer {
private:
    virtual void onTick() const override { /* ... */ }
};
// This says Widget is-a Timer, which is wrong!

// [GOOD] Option 2: Private inheritance
class Widget : private Timer {
private:
    virtual void onTick() const override { /* ... */ }
};

// [GOOD] Option 3: Composition (usually better!)
class Widget {
private:
    class WidgetTimer : public Timer {
    public:
        virtual void onTick() const override;
        // ...
    };
    WidgetTimer timer;
};
```

**When to Use Private Inheritance**:

```cpp
// 1. When you need access to protected members
class Base {
protected:
    void protectedFunc();
};

class Derived : private Base {
    void foo() {
        protectedFunc();  // OK with private inheritance
    }
};

// 2. When you need to override virtual functions
// 3. When dealing with empty base optimization (EBO)

class Empty {};  // sizeof(Empty) == 1 (usually)

class HoldsInt {
private:
    int x;
    Empty e;  // Probably takes up space
};
// sizeof(HoldsInt) probably > sizeof(int)

class HoldsInt2 : private Empty {
private:
    int x;
};
// sizeof(HoldsInt2) probably == sizeof(int) (EBO)
```

## 6.9 Item 40: Use multiple inheritance judiciously

**Concept**: Multiple inheritance (MI) is more complex than single inheritance but can be useful in some situations.

**Deadly Diamond of Death**:

```
class File {
public:
    std::string filename() const;
};

class InputFile : public File { /* ... */ };
class OutputFile : public File { /* ... */ };

class IOFile : public InputFile, public OutputFile {
    // Problem: Two copies of File!
    // Two filename() functions!
};

// [GOOD] SOLUTION: Virtual inheritance
class File { /* ... */ };

class InputFile : virtual public File { /* ... */ };
class OutputFile : virtual public File { /* ... */ };

class IOFile : public InputFile, public OutputFile {
    // Only one copy of File
};
```

**Cost of Virtual Inheritance**:

```
// Virtual inheritance has costs:
// - Larger objects
// - Slower access to virtual base members
// - More complex initialization

// Rules for virtual base initialization:
// - Initialization responsibility assigned to most derived class
// - Must know about virtual bases even if far away in hierarchy

class IPerson {
public:
    virtual ~IPerson();
    virtual std::string name() const = 0;
    virtual std::string birthDate() const = 0;
};

class DatabaseID {
public:
    virtual ~DatabaseID();
    virtual std::string dbID() const = 0;
};

// MI for interface:
class PersonInfo : public IPerson, public DatabaseID {
public:
    virtual std::string name() const override {
        return theName;
    }
```

```
    virtual std::string birthDate() const override {
        return theBirthDate;
    }

    virtual std::string dbID() const override {
        return theDBID;
    }

private:
    std::string theName;
    std::string theBirthDate;
    std::string theDBID;
};
```

**Reasonable MI Example**:

```
// Combine interface and implementation
class IPerson {
public:
    virtual ~IPerson();
    virtual std::string name() const = 0;
};

class PersonInfo {
public:
    explicit PersonInfo(const std::string& n) : theName(n) {}

    std::string name() const {
        return theName;
    }

private:
    std::string theName;
};

// Use MI to combine interface and implementation
class CPerson : public IPerson, private PersonInfo {
public:
    explicit CPerson(const std::string& name) : PersonInfo(name) {}

    virtual std::string name() const override {
        return PersonInfo::name();
    }
};
```

---

# 7 Chapter 7: Templates and Generic Programming

## 7.1 Item 41: Understand implicit interfaces and compile-time polymorphism

**Concept**: Templates support implicit interfaces and compile-time polymorphism, unlike OOP's explicit interfaces and runtime polymorphism.

**Example**:

```cpp
// OOP: Explicit interface, runtime polymorphism
class Widget {
public:
    Widget();
    virtual ~Widget();
    virtual std::size_t size() const;
    virtual void normalize();
    void swap(Widget& other);
};

void doProcessing(Widget& w) {
    if (w.size() > 10) {  // Explicit interface
        Widget temp(w);
        temp.normalize();
        temp.swap(w);
    }
}
// w must support Widget interface
// Some calls may be virtual (runtime polymorphism)

// Templates: Implicit interface, compile-time polymorphism
template<typename T>
void doProcessing(T& w) {
    if (w.size() > 10) {  // Implicit interface
        T temp(w);
        temp.normalize();
        temp.swap(w);
    }
}
// T must support:
// - size() returning something comparable to int
// - Copy constructor
// - normalize()
// - swap()
// Which T is used determined at compile time
```

## 7.2 Item 42: Understand the two meanings of typename

**Concept**: `typename` is used to specify template parameters and to identify nested dependent type names.

**Example**:

```cpp
// Use 1: Template parameter
template<typename T>
class Widget;

template<class T>  // Same as typename here
class Widget;

// Use 2: Nested dependent type names
template<typename C>
void print2nd(const C& container) {
    if (container.size() >= 2) {
        C::const_iterator iter(container.begin());  // [BAD] Error!
        ++iter;
        std::cout << *iter;
    }
}

// Problem: C::const_iterator is a dependent name
```

```cpp
// Compiler doesn't know if it's a type or a static member

template<typename C>
void print2nd(const C& container) {
    if (container.size() >= 2) {
        typename C::const_iterator iter(container.begin());  // [GOOD] Correct
        ++iter;
        std::cout << *iter;
    }
}
```

**When typename is Required**:

```cpp
template<typename C>
void f(const C& container) {
    typename C::iterator iter;  // Required: nested dependent type
}

// Exception: Not in base class list or initialization list
template<typename T>
class Derived : public Base<T>::Nested {  // No typename
public:
    explicit Derived(int x)
        : Base<T>::Nested(x) {  // No typename
        typename Base<T>::Nested temp;  // typename required
    }
};
```

## 7.3 Item 43: Know how to access names in templatized base classes

**Concept**: In template inheritance, names in base classes are not visible by default. You must explicitly make them visible.

**Example**:

```cpp
class CompanyA {
public:
    void sendCleartext(const std::string& msg);
    void sendEncrypted(const std::string& msg);
};

class CompanyB {
public:
    void sendCleartext(const std::string& msg);
    void sendEncrypted(const std::string& msg);
};

class MsgInfo { /* ... */ };

template<typename Company>
class MsgSender {
public:
    void sendClear(const MsgInfo& info) {
        std::string msg;
        // Create msg from info
        Company c;
        c.sendCleartext(msg);
    }

    void sendSecret(const MsgInfo& info) { /* ... */ }
};
```

```cpp
template<typename Company>
class LoggingMsgSender : public MsgSender<Company> {
public:
    void sendClearMsg(const MsgInfo& info) {
        // Log before
        sendClear(info);  // [BAD] Error! Name not found!
        // Log after
    }
};

// Problem: Compiler doesn't look in base class
// Because base class might be specialized

// Total specialization example:
template<>
class MsgSender<CompanyZ> {
public:
    void sendSecret(const MsgInfo& info) { /* ... */ }
    // No sendClear!
};
```

**Solutions**:

```cpp
// [GOOD] SOLUTION 1: this->
template<typename Company>
class LoggingMsgSender : public MsgSender<Company> {
public:
    void sendClearMsg(const MsgInfo& info) {
        this->sendClear(info);  // OK
    }
};

// [GOOD] SOLUTION 2: using declaration
template<typename Company>
class LoggingMsgSender : public MsgSender<Company> {
public:
    using MsgSender<Company>::sendClear;  // Tell compiler to assume it's there

    void sendClearMsg(const MsgInfo& info) {
        sendClear(info);  // OK
    }
};

// [GOOD] SOLUTION 3: Explicit qualification
template<typename Company>
class LoggingMsgSender : public MsgSender<Company> {
public:
    void sendClearMsg(const MsgInfo& info) {
        MsgSender<Company>::sendClear(info);  // OK but disables virtual
    }
};
```

## 7.4 Item 44: Factor parameter-independent code out of templates

**Concept**: Templates can lead to code bloat. Extract parameter-independent code into non-template base classes.

**Example**:

```cpp
// [BAD] BAD: Code bloat
template<typename T, std::size_t n>
class SquareMatrix {
public:
    void invert() {
        // Matrix inversion code for n x n matrix
    }
};

SquareMatrix<double, 5> sm1;
SquareMatrix<double, 10> sm2;
sm1.invert();  // Instantiates invert for 5x5
sm2.invert();  // Instantiates invert for 10x10
// Two copies of invert code!

// [GOOD] BETTER: Factor out parameter-independent code
template<typename T>
class SquareMatrixBase {
protected:
    SquareMatrixBase(std::size_t n, T* pMem)
        : size(n), pData(pMem) {}

    void invert(std::size_t matrixSize);  // Size as parameter

private:
    std::size_t size;
    T* pData;
};

template<typename T, std::size_t n>
class SquareMatrix : private SquareMatrixBase<T> {
public:
    SquareMatrix()
        : SquareMatrixBase<T>(n, data) {}

    void invert() {
        this->invert(n);  // Call base class version
    }

private:
    T data[n * n];
};

// Now only one invert per type, not per size!
```

## 7.5 Item 45: Use member function templates to accept “all compatible types”

**Concept**: Member templates allow conversion between related template instantiations.

**Example**:

```cpp
// Smart pointer that accepts conversions like raw pointers
template<typename T>
class SmartPtr {
public:
    explicit SmartPtr(T* realPtr);

    // Member template for constructor
    template<typename U>
    SmartPtr(const SmartPtr<U>& other)
        : heldPtr(other.get()) {  // Implicit conversion from U* to T*
    }

    T* get() const { return heldPtr; }

private:
    T* heldPtr;
};

// Usage:
class Top { /* ... */ };
class Middle : public Top { /* ... */ };
class Bottom : public Middle { /* ... */ };

SmartPtr<Top> pt1 = SmartPtr<Middle>(new Middle);  // OK
SmartPtr<Top> pt2 = SmartPtr<Bottom>(new Bottom);  // OK
SmartPtr<const Top> pct2 = pt1;  // OK: non-const to const

// SmartPtr<Bottom> pb = SmartPtr<Top>(new Top);  // Error! (correct)

// Also for assignment:
template<typename T>
class SmartPtr {
public:
    template<typename U>
    SmartPtr& operator=(const SmartPtr<U>& other) {
        heldPtr = other.get();
        return *this;
    }
};
```

**Member Templates Don't Replace Compiler-Generated Functions**:

```cpp
template<typename T>
class SmartPtr {
public:
    // Member template (generalized copy constructor)
    template<typename U>
    SmartPtr(const SmartPtr<U>& other);

    // Still need regular copy constructor
    SmartPtr(const SmartPtr& other);  // Compiler generates if not declared
};

// Declare both explicitly:
template<typename T>
class shared_ptr {
public:
    shared_ptr(const shared_ptr& r);  // Copy constructor

    template<typename Y>
    shared_ptr(const shared_ptr<Y>& r);  // Generalized copy constructor
```

```
    shared_ptr& operator=(const shared_ptr& r);  // Copy assignment

    template<typename Y>
    shared_ptr& operator=(const shared_ptr<Y>& r);  // Generalized assignment
};
```

## 7.6 Item 46: Define non-member functions inside templates when type conversions are desired

**Concept**: For template operators that should support implicit conversions on all arguments, define as friend functions inside the class template.

**Example**:

```
template<typename T>
class Rational {
public:
    Rational(const T& numerator = 0, const T& denominator = 1);
    const T numerator() const;
    const T denominator() const;
};

// [BAD] Problem: Won't compile for mixed-mode operations
template<typename T>
const Rational<T> operator*(const Rational<T>& lhs, const Rational<T>& rhs) {
    return Rational<T>(lhs.numerator() * rhs.numerator(),
                       lhs.denominator() * rhs.denominator());
}

Rational<int> oneHalf(1, 2);
Rational<int> result = oneHalf * 2;  // Error!
// Template argument deduction fails for 2

// [GOOD] SOLUTION: Friend function inside class
template<typename T>
class Rational {
public:
    Rational(const T& numerator = 0, const T& denominator = 1);

    friend const Rational operator*(const Rational& lhs, const Rational& rhs) {
        return Rational(lhs.numerator() * rhs.numerator(),
                        lhs.denominator() * rhs.denominator());
    }
};

// Now works:
Rational<int> oneHalf(1, 2);
Rational<int> result = oneHalf * 2;  // OK! 2 converted to Rational<int>
```

**For Complex Implementations**:

```cpp
template<typename T>
class Rational {
public:
    friend const Rational operator*(const Rational& lhs, const Rational& rhs) {
        return doMultiply(lhs, rhs);  // Call helper
    }
};

// Helper function template
template<typename T>
const Rational<T> doMultiply(const Rational<T>& lhs, const Rational<T>& rhs) {
    return Rational<T>(lhs.numerator() * rhs.numerator(),
                       lhs.denominator() * rhs.denominator());
}
```

## 7.7 Item 47: Use traits classes for information about types

**Concept**: Traits provide compile-time information about types.

**Example**:

```cpp
// STL iterator categories
struct input_iterator_tag {};
struct output_iterator_tag {};
struct forward_iterator_tag : public input_iterator_tag {};
struct bidirectional_iterator_tag : public forward_iterator_tag {};
struct random_access_iterator_tag : public bidirectional_iterator_tag {};

// Iterator traits
template<typename IterT>
struct iterator_traits {
    typedef typename IterT::iterator_category iterator_category;
};

// Partial specialization for pointers
template<typename T>
struct iterator_traits<T*> {
    typedef random_access_iterator_tag iterator_category;
};

// Usage: advance function
template<typename IterT, typename DistT>
void advance(IterT& iter, DistT d) {
    doAdvance(iter, d,
              typename std::iterator_traits<IterT>::iterator_category());
}

// Implementation for different categories
template<typename IterT, typename DistT>
void doAdvance(IterT& iter, DistT d, std::random_access_iterator_tag) {
    iter += d;  // Fast for random access
}

template<typename IterT, typename DistT>
void doAdvance(IterT& iter, DistT d, std::bidirectional_iterator_tag) {
    if (d >= 0) {
        while (d--) ++iter;
    } else {
        while (d++) --iter;
    }
```

```
}

template<typename IterT, typename DistT>
void doAdvance(IterT& iter, DistT d, std::input_iterator_tag) {
    if (d < 0) {
        throw std::out_of_range("Negative distance");
    }
    while (d--) ++iter;
}
```

## 7.8 Item 48: Be aware of template metaprogramming

**Concept**: Template metaprogramming (TMP) performs computation at compile time, shifting work from runtime to compile time.

**Example**:

```
// Factorial at compile time
template<unsigned n>
struct Factorial {
    enum { value = n * Factorial<n - 1>::value };
};

template<>
struct Factorial<0> {
    enum { value = 1 };
};

int main() {
    std::cout << Factorial<5>::value;  // Prints 120
    std::cout << Factorial<10>::value; // Prints 3628800
    // Computed at compile time!
}

// Type selection at compile time
template<bool condition, typename T, typename F>
struct IF {
    typedef T type;
};

template<typename T, typename F>
struct IF<false, T, F> {
    typedef F type;
};

// Usage:
IF<sizeof(int) == 4, int, long>::type i;  // int if sizeof(int)==4, else long

// Modern C++11 equivalent: std::conditional
std::conditional<sizeof(int) == 4, int, long>::type i;
```

**TMP Example: Type Traits**:

```cpp
// Remove const from type
template<typename T>
struct remove_const {
    typedef T type;
};

template<typename T>
struct remove_const<const T> {
    typedef T type;
};

// Usage:
remove_const<const int>::type x;  // x is int, not const int

// STL provides std::remove_const, std::is_const, etc.
```

---

# 8 Chapter 8: Customizing new and delete

## 8.1 Item 49: Understand the behavior of the new-handler

**Concept**: When operator new can't allocate memory, it calls a client-specified error-handling function (new-handler) before throwing bad_alloc.

**Example**:

```cpp
namespace std {
    typedef void (*new_handler)();
    new_handler set_new_handler(new_handler p) throw();
}

// Custom new-handler
void outOfMem() {
    std::cerr << "Unable to allocate memory!" << std::endl;
    std::abort();
}

int main() {
    std::set_new_handler(outOfMem);

    int* pBigDataArray = new int[100000000000L];  // Will fail
    // Calls outOfMem before throwing
}
```

**What a new-handler Should Do**: 1. **Make more memory available** (release a reserve) 2. **Install a different new-handler** (that might succeed) 3. **Deinstall the new-handler** (pass null to set_new_handler) 4. **Throw an exception** (bad_alloc or derived) 5. **Not return** (abort or exit)

**Class-Specific new-handlers**:

```cpp
class Widget {
public:
    static std::new_handler set_new_handler(std::new_handler p) throw();
    static void* operator new(std::size_t size) throw(std::bad_alloc);

private:
    static std::new_handler currentHandler;
};

std::new_handler Widget::currentHandler = 0;

std::new_handler Widget::set_new_handler(std::new_handler p) throw() {
    std::new_handler oldHandler = currentHandler;
    currentHandler = p;
    return oldHandler;
}

// RAII class for new-handler management
class NewHandlerHolder {
public:
    explicit NewHandlerHolder(std::new_handler nh)
        : handler(nh) {}

    ~NewHandlerHolder() {
        std::set_new_handler(handler);
    }

private:
    std::new_handler handler;
    NewHandlerHolder(const NewHandlerHolder&);  // Prevent copying
    NewHandlerHolder& operator=(const NewHandlerHolder&);
};

void* Widget::operator new(std::size_t size) throw(std::bad_alloc) {
    NewHandlerHolder h(std::set_new_handler(currentHandler));
    return ::operator new(size);  // Use global operator new
}  // Restore global new-handler

// Usage:
void outOfMemForWidget() {
    std::cerr << "Widget allocation failed!" << std::endl;
    std::abort();
}

Widget::set_new_handler(outOfMemForWidget);
Widget* pw1 = new Widget;  // If fails, calls outOfMemForWidget

std::string* ps = new std::string;  // If fails, calls global new-handler
```

## 8.2 Item 50: Understand when it makes sense to replace new and delete

**Reasons to Replace new and delete**: 1. **Detect usage errors** (overruns, underruns, double deletes) 2. **Collect statistics** (allocation patterns, lifetimes) 3. **Increase performance** (custom allocators for specific types) 4. **Reduce memory overhead** (general-purpose allocators waste space) 5. **Improve locality** (place related objects near each other) 6. **Obtain non-traditional behavior** (shared memory, persistent storage)

**Example**:

```
// Custom operator new to detect overruns/underruns
static const int signature = 0xDEADBEEF;

typedef unsigned char Byte;

void* operator new(std::size_t size) throw(std::bad_alloc) {
    std::size_t realSize = size + 2 * sizeof(int);

    void* pMem = malloc(realSize);
    if (!pMem) throw std::bad_alloc();

    // Write signatures at start and end
    *(static_cast<int*>(pMem)) = signature;
    *(reinterpret_cast<int*>(static_cast<Byte*>(pMem) + realSize - sizeof(int))) = signature;

    // Return pointer to memory just past first signature
    return static_cast<Byte*>(pMem) + sizeof(int);
}

void operator delete(void* pMemory) throw() {
    if (pMemory == 0) return;

    // Get actual start of allocation
    void* pMem = static_cast<Byte*>(pMemory) - sizeof(int);

    // Check signatures
    if (*(static_cast<int*>(pMem)) != signature) {
        std::cerr << "Underrun detected!" << std::endl;
    }

    // Check end signature (simplified)
    // ...

    free(pMem);
}
```

**Issues to Consider**:

```
// Alignment issues
void* operator new(std::size_t size) throw(std::bad_alloc) {
    void* pMem = malloc(size);
    // malloc returns properly aligned memory
    // Custom allocators must ensure proper alignment!
    return pMem;
}

// Most platforms require double alignment (8 bytes)
// Some types require 16-byte alignment
```

## 8.3 Item 51: Adhere to convention when writing new and delete

**Conventions for operator new**: 1. Return correct value 2. Call new-handler when insufficient memory 3. Handle zero-byte requests 4. Avoid hiding “normal” form

**Example**:

```cpp
void* operator new(std::size_t size) throw(std::bad_alloc) {
    if (size == 0) {
        size = 1;  // Handle 0-byte requests
    }

    while (true) {
        // Attempt to allocate size bytes
        void* p = malloc(size);

        if (p) {
            return p;  // Success
        }

        // Allocation failed; find current new-handler
        std::new_handler handler = std::set_new_handler(0);
        std::set_new_handler(handler);

        if (handler) {
            (*handler)();  // Call it
        } else {
            throw std::bad_alloc();  // No handler; throw
        }
    }
}
```

**Class-specific operator new**:

```cpp
class Base {
public:
    static void* operator new(std::size_t size) throw(std::bad_alloc);
};

class Derived : public Base {
    // Inherits operator new
};

void* Base::operator new(std::size_t size) throw(std::bad_alloc) {
    if (size != sizeof(Base)) {  // Wrong size: derived class
        return ::operator new(size);  // Use standard operator new
    }

    // Otherwise, handle allocation here
    // ...
}
```

**Conventions for operator delete**: 1. Deleting null is safe 2. Handle size correctly for derived classes

**Example**:

```cpp
void operator delete(void* rawMemory) throw() {
    if (rawMemory == 0) return;  // Do nothing if null pointer

    // Deallocate memory
    free(rawMemory);
}

// Class-specific
class Base {
public:
    static void* operator new(std::size_t size) throw(std::bad_alloc);
    static void operator delete(void* rawMemory, std::size_t size) throw();
};

void Base::operator delete(void* rawMemory, std::size_t size) throw() {
    if (rawMemory == 0) return;

    if (size != sizeof(Base)) {  // Wrong size: derived class
        ::operator delete(rawMemory);
        return;
    }

    // Deallocate memory pointed to by rawMemory
    // ...
}
```

## 8.4 Item 52: Write placement delete if you write placement new

**Concept**: If a constructor throws after placement new allocates memory, the runtime looks for a matching placement delete. If not found, no delete is called—memory leak!

**Example**:

```cpp
class Widget {
public:
    // Normal operator new
    static void* operator new(std::size_t size) throw(std::bad_alloc);

    // Placement operator new (with ostream for logging)
    static void* operator new(std::size_t size, std::ostream& logStream) throw(std::bad_alloc)

    // Normal operator delete
    static void operator delete(void* pMemory) throw();

    // Placement operator delete (matches placement new)
    static void operator delete(void* pMemory, std::ostream& logStream) throw();
};

Widget* pw = new Widget;  // Calls normal operator new
delete pw;                // Calls normal operator delete

Widget* pw2 = new (std::cerr) Widget;  // Calls placement new
// If Widget constructor throws, calls placement delete
delete pw2;  // Calls normal operator delete (NOT placement delete!)
```

**Name Hiding with new/delete**:

```cpp
class Base {
public:
    static void* operator new(std::size_t size) throw(std::bad_alloc);
};

class Derived : public Base {
public:
    static void* operator new(std::size_t size, std::ostream& logStream) throw(std::bad_alloc)
};

Derived* p = new Derived;        // [BAD] Error! Normal new hidden
Derived* p2 = new (std::cerr) Derived;  // OK

// [GOOD] Solution: Make all forms available
class Derived : public Base {
public:
    using Base::operator new;  // Make Base's versions visible

    static void* operator new(std::size_t size, std::ostream& logStream) throw(std::bad_alloc)
};
```

**Standard Forms to Provide**:

```cpp
// Normal new/delete
void* operator new(std::size_t) throw(std::bad_alloc);
void operator delete(void*) throw();
void operator delete(void*, std::size_t) throw();

// Placement new/delete (no-throw)
void* operator new(std::size_t, void*) throw();
void operator delete(void*, void*) throw();

// Nothrow new/delete
void* operator new(std::size_t, const std::nothrow_t&) throw();
void operator delete(void*, const std::nothrow_t&) throw();
```

---

# 9 Chapter 9: Miscellany

## 9.1 Item 53: Pay attention to compiler warnings

**Concept**: Take compiler warnings seriously. Different compilers warn about different things, so strive to compile warning-free on multiple compilers.

**Example**:

```cpp
class B {
public:
    virtual void f() const;
};

class D : public B {
public:
    virtual void f();  // Warning: hides B::f()
};

// Intent was probably to override, but forgot const
// Correct version:
class D : public B {
public:
    virtual void f() const override;  // Use override in C++11+
};
```

## 9.2 Item 54: Familiarize yourself with the standard library

**Key Components**:

1. 
2. 
3. 
4. 
5. 
6. 

**Example**:

```cpp
#include <vector>
#include <algorithm>
#include <iostream>
#include <numeric>

int main() {
    std::vector<int> v = {5, 2, 8, 1, 9};

    // Sort
    std::sort(v.begin(), v.end());

    // Find
    auto it = std::find(v.begin(), v.end(), 8);
    if (it != v.end()) {
        std::cout << "Found: " << *it << std::endl;
    }

    // Transform
    std::vector<int> doubled(v.size());
    std::transform(v.begin(), v.end(), doubled.begin(),
                   [](int x) { return x * 2; });

    // Accumulate
    int sum = std::accumulate(v.begin(), v.end(), 0);

    return 0;
}
```

## 9.3 Item 55: Familiarize yourself with Boost

**What is Boost?** - Peer-reviewed, open-source C++ libraries - Many Boost libraries became part of C++11/14/17/20 standards - Covers areas not in standard library

**Important Boost Libraries**:

1.
2.
3.
4.
5.
6.
7.
8.
9.
10.

**Example**:

```
// Boost.Bind example (similar to std::bind in C++11)
#include <boost/bind.hpp>
#include <algorithm>
#include <vector>

bool isLessThan(int x, int threshold) {
    return x < threshold;
}

std::vector<int> v = {1, 5, 3, 8, 2};
int count = std::count_if(v.begin(), v.end(),
                          boost::bind(isLessThan, _1, 5));

// Boost.Filesystem example
#include <boost/filesystem.hpp>

namespace fs = boost::filesystem;

void listFiles(const fs::path& dir) {
    if (fs::exists(dir) && fs::is_directory(dir)) {
        for (auto& entry : fs::directory_iterator(dir)) {
            std::cout << entry.path().filename() << std::endl;
        }
    }
}
```

---

# 10 Summary of Key Principles

## 10.1 Design Principles

1.
2.
3.
4.
5.

## 10.2 Class Design

1.
2.

3.
4.
5.

## 10.3 Constructors and Destructors

1.
2.
3.
4.
5.

## 10.4 Inheritance

1.
2.
3.
4.
5.

## 10.5 Templates

1.
2.
3.
4.
5.

## 10.6 Resource Management

1.
2.
3.
4.
5.

## 10.7 Exception Safety

1.
2.
3.
4.

## 10.8 Efficiency

1.
2.
3.
4.
5.

## 10.9 Best Practices

1.
2.
3.
4.
5.

# 11 Quick Reference: C++ Best Practices Checklist

## 11.1 Before Writing a Class

- 
- 
- 
- 
- 

## 11.2 Writing a Class

- 
- 
- 
- 
- 
- 
- 

## 11.3 Using Inheritance

- 
- 
- 
- 
- 
- 

## 11.4 Templates

- 
- 
- 
- 

## 11.5 Resource Management

- 
- 
- 
- 
- 

## 11.6 Performance

- 
- 
- 
- 
- 

---

# 12 Conclusion

Effective C++ programming requires understanding the language's subtleties and applying best practices consistently. The principles outlined in this guide help you:

- 

- 
- 
- 
- 

Remember: - **The compiler is your friend**: Use it to catch errors early - **Const correctness matters**: It documents intent and catches bugs - **RAII is fundamental**: Let objects manage resources - **Understand what the compiler generates**: Know when to rely on defaults - **Make interfaces intuitive**: Easy to use correctly, hard to use incorrectly

Continue learning by: - Reading the actual “Effective C++” book by Scott Meyers - Exploring “More Effective C++” and “Effective Modern C++” - Practicing these principles in real code - Reviewing and refactoring existing code with these guidelines

---

**Document Information** - Based on concepts from: “Effective C++” by Scott Meyers (Third Edition) - Original examples and explanations created for educational purposes - Updated with modern C++11/14/17 features where applicable - Created: 2026-04-09

---

*This document is an educational resource inspired by Scott Meyers’ work. For the complete and authoritative treatment, please refer to the original “Effective C++” books.* “‘